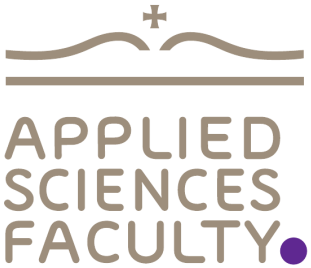
C++ — бітові маніпуляції





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- Маніпуляції бітами важливі для багатьох задач – роботи із мікроконтролерами, бінарними даними, (зокрема – архівами), мережевими пакетами тощо. Також часто використовують у прикладному API для керування набором опцій.
- Приклад: на архітектурі AVR8, регістри DDRD, PORTD, PIND керують вісьмома пінами порта D, по біту на пін.
- Сучасні процесори мають багато зручних команд для цього, хоча, зазвичай, доводиться покладатися на здатність компілятора їх ефективно використати.
- C та C++ мають цілий набір побітових операцій – побітові AND, OR, XOR, NOT.
- Також важливими є операції бітового зсуву.
- Так як для знакових чисел є нюанси (див. далі), бажано організовувати код так, щоб всі бітові маніпуляції та зсуви відбувалися над беззнаковими числами.
- C++14 дозволив використовувати бінарні літерали:

  const uint32_t bit32_3b = 0b0000'1000;

- До того слід було користуватися або шістнадцятковим представленням (насправді – зручнішим через лаконічність, але трохи важчим у читанні), або бітовими зсувами:

  const uint32_t bit32_3a = 1 << 3;
  const uint32_t cbit32_3 = 0x8;

- Із знаковими числами складніше – хоча всі сучасні архітектури (крім, можливо, екзотичних DSP) використовують доповнювальний код, стандарт не зобов'язує їх до цього[1].
- Незалежно від бітового представлення, гарантується, що перетворення із від'ємного signed-типу у беззнаковий того ж розміру відбувається в доповнювальному коді. (Стандарт 2017, [conv.integral].2).
- Надалі вважатимемо, що апаратне забезпечення працює в доповнювальному коді. Тоді слід пам'ятати про знакове розширення – при збільшенні розрядності (копіюванні у тип із більшою кількістю бітів), для додатних чисел у нові старші біти клонуватимуться нулі, для від'ємних – одинички.
- Наприклад uint32_t завжди має 32 біти. На машині, де int – 32-бітний, такий код:

  uint32_t x32 = −1u;

  справді встановить всі біти в одиницю.
- А ось такий:

  uint64_t x64 = −1u;

  встановить лише молодші 32 біти!

[1] І деякі компілятори можуть із цього користатися: http://indrekis2.blogspot.com/2013/04/cc.html

Невизначена та системозалежна поведінка

- Не залежно від знаковості, побітові оператори працюють над бітовим представленням кожного із операндів, після звичних приведень типів. (Див. стандарт, [expr.bit.and] і т.д.).
- З бітовими зсувами трохи складніше.
- Зсув на від'ємну величину чи на величину, більшу або рівну кількості бітів аргументу – **невизначена поведінка**.
- Зсув У знаковий біт (зсув ліворуч) – **невизначена поведінка**.
- Зсув із знакового біта – implementation-defined. (Див. стандарт, [expr.shift]). MSCV на x8632/64[2] клонує знаковий біт, MSVC for ARM веде себе трохи складніше[3]. Так виглядає, що GCC (див. пункт 4.5, "C Implementation-Defined Behavior"-"Integers") просто вважає знаковий біт просто старшим бітом.
- C++20 обіцяє (`http://eel.is/c++draft/expr.shift`): зсув у чи з знакового біта чітко визначений: зсув ліворуч веде себе як беззнаковий, а зсув праворуч: "Right-shift on signed integral types is an arithmetic right shift, which performs sign-extension" .
- В C ситуація трохи інша, особливо між C90 і C99 – див. відповідні стандарти.

[2] `https://docs.microsoft.com/en-us/cpp/cpp/left-shift-and-right-shift-operators-input-and-output?view=vs-2019`.

[3] Див. `https://devblogs.microsoft.com/cppblog/hello-arm-exploring-undefined-unspecified-and-implementation-defined-behavior-in-c/`.

- Бітовий зсув ліворуч[4] на N біт множить число на 2^N.
- Бітовий зсув праворуч – ділить на 2^N.
- В старих книгах рекомендують такі оптимізації. Однак, із сучасними оптимізаторами їх не варто робити – компілятор сам справиться із цією задачею краще[5].

[4] Не відволікаючись зараз на подробиці із попереднього слайда.

[5] Не абсолютно завжди, але в абсолютній більшості випадків

Встановлення біта (set)

- Встановити біт із номером n змінної x в 1, не зачіпаючи решту:

  x |= 1UL<<n;

- На деяких архітектурах семантично еквівалентна операція може генерувати більше машинних команд:

  x = x | (1UL<<n);

- Встановлювати так можна і декілька бітів зразу:

  x |= (1UL << 3 | 1UL << 4);
  x |= 0b1101;

- Правий операнд називають бітовою маскою – вона задає патерн операцій над бітами.
- Суфікс UL потрібен, щоб вираз праворуч був не меншим за unsigned long, без цього можуть бути сюрпризи. Якщо unsigned long замало – використовуйте ULL.

Очищення біта (clear)

- x &= ~mask;

Зміна біта на протилежний (toggle)

- x ^= mask;

Перевірка біта

- **bool** is_set = x & mask;
- Один біт, із номером n, можна також перевірити так:

  bool is_set = (x>>n) & 1U;

Встановити в n-й біт значення з src

- Якщо src строго 1 або 0:

  x ^= (−src ^ x) & (1UL << n);

- Якщо ж ні, його можна привести до такої форми операцією !!:

  unsigned long src_as_bool = !!src;
  x ^= (−src_as_bool ^ x) & (1UL << n);

- Можна також простіше – очищаємо, потім встановлюємо (src – 0 або 1).

  x = (x & ~(1UL << n)) | (src << n);

- "Bit twiddling hacks" (див. далі) стверджує, що таке рішення на сучасних процесорах може бути швидшим:

  x = (x & ~(1UL << n)) | (−src & (1UL << n));

"Bit twiddling hacks"

- `http://graphics.stanford.edu/~seander/bithacks.html` містить багато більш просунутих бітових трюків, із детальним аналізом їх коректності, продуктивності та портабельності.
- Розглянемо тут частину прикладів звідти.
- Пам'ятаємо, що розгалуження (branching) всілякі – if і т.д., навіть замасковані тернарним оператором, можуть сильно сповільнювати сучасні процесори – через довгі конвеєри, якщо передбачення переходів не вдається.
- Замість unsigned int вартувало б писати `uint32\_t` – код написано із розрахунком на 32-бітний int. Але лінь...
- Автор іноді пропонує розгортати цикли (loop unrolling) вручну. Для сучасних компіляторів це не обов'язково, а іноді – погана ідея, вони змушені згортати таке ручне розгортання назад, щоб потім згорнути по своєму. Хоча, іноді може й дати виграш. Вимірюйте!
- Також, автор іноді вживає ключове слово register, що вже дуже давно позбавлено сенсу.

УВАГА! Нижче вважається, що int – 32-бітний!

Зокрема, для AVR доведеться модифікувати.

Визначення знаку int

- Результат – 0 або -1.

```
int v;                  // Число, що аналізуємо
int sign;               // Тут результат, 0 або -1

sign = -(v < 0);  // Може бути розгалуження

sign = -(int)((unsigned int)((int)v) >>
                    (sizeof(int) * CHAR_BIT - 1));
                    // А так - точно не буде.
```

- Результат – +1 або -1, не портабельно до C++20.

```
sign = +1 | (v >> (sizeof(int) * CHAR_BIT - 1));
```

- Результат – +1, 0 або -1:

```
sign = (v > 0) - (v < 0);
sign = (v != 0) | -(int)((unsigned int)((int)v) >>
                        (sizeof(int) * CHAR_BIT - 1));
```

- Результат – 0 для від'ємних, 1 для додатних:

```
sign = 1 ^ ((unsigned int)v >> (sizeof(int) * CHAR_BIT - 1));
```

Перевірка, чи два числа мають різний знак

- **int** x, y;

  bool f = ((x ^ y) < 0);

Обчислення модуля цілого числа без розгалужень

- **int** v;
  unsigned int r;

```
int const mask = v >> sizeof(int) * CHAR_BIT - 1;
                                        // Не портабельно до C++20

r = (v + mask) ^ mask;

r = (v ^ mask) - mask; // Потенційно запатентовано
```

Обчислення мінімуму чи максимуму без розгалужень

- **int** x, y;
  int res_min, res_max;

  res_min = y ^ ((x ^ y) & −(x < y)); // *min(x, y)*
  res_max = x ^ ((x ^ y) & −(x < y)); // *max(x, y)*

  //! Хоча, якщо є *conditional move*, так, певне, краще:
  res_min = (x < y) ? x : y,
  //! Тим більше, що $x < y$ теж може створити розгалуження.

Перевірка, чи ціле число є степенем двійки – має лише один одиничний біт

- **unsigned int** v;
  bool res;

  res = (v & (v − 1)) == 0; // Буде *true* для *0*

  // Виправляємо:
  res = v && !(v & (v − 1));

Розширення знаку для числа фіксованої ширини

- Елегантний трюк – давайте, хай тим компілятор займеться:

```
int x; // Тут зберігається бітове5— знакове число
int res;
struct {signed int x:5;} s;
res = s.x = x;

//! Або так:
template <typename T, unsigned B>
inline T signextend(const T x){
  struct {T x:B;} s;
  return s.x = x;
}

int res = signextend<signed int,5>(x);
```

Розширення знаку для числа невідомої наперед ширини

- Можна якось так

```
unsigned b; // Скільки бітів є в числі x
int  x;
int  res;
int  const m = 1U << (b - 1);
x = x & ((1U << b) - 1);  // Зануляємо біти із номерами > b.
                          // Не потрібно, якщо і так нульові.
r = (x ^ m) - m;
```

- За іншими варіантами див. оригінальне посилання, `http://graphics.stanford.edu/~seander/bithacks.html#VariableSignExtend`.

Зміна знаку згідно умови, без розгалужень

- **bool** fDontNegate; // *true* —– не змінювати знак
  int val;
  int res;

  res = (fDontNegate ^ (fDontNegate − 1)) ∗ val;

  bool fNegate; // *true* —– змінювати знак

  res = (val ^ −fNegate) + fNegate;

Об'єднання бітів із двох джерел згідно маски

- **unsigned int** a; // звідси брати незамасковані біти
  unsigned int b; // a звідси —— замасковані
  unsigned int mask; // *1*, якщо біт слід взяти з *b*, *0* —— якщо з *a*
  unsigned int res;

  res = a ^ ((a ^ b) & mask);

Підрахунок бітів

- Наївний підхід:

```
unsigned int v; // Рахуємо, скільки тут одиничних біт
unsigned int cnt; // Результат

for (cnt = 0; v; v >>= 1)
{
  cnt += v & 1;
}
```

Виконує завжди стільки ітерацій, скільки біт є в unsigned int.

- Метод Кернігана:

```
for (cnt = 0; v; cnt++)
{
  v &= v - 1; // clear the least significant bit set
}
```

Кількість ітерацій рівна кількості встановлених біт.

Підраховуємо за допомогою таблиці перетворень (lookup table)

- Різних комбінацій бітів у байті всього 256 – можна наперед порахувати, скільки у кожній із них бітів та користуватися цією табличкою. Мінус – зростання тиску на кеш.
- Готуємо табличку:

```
static const unsigned char BitsSetTable256[256] = {
#   define B2(n) n,     n+1,     n+1,     n+2
#   define B4(n) B2(n), B2(n+1), B2(n+1), B2(n+2)
#   define B6(n) B4(n), B4(n+1), B4(n+1), B4(n+2)
    B6(0), B6(1), B6(1), B6(2)
#undef B2
#undef B4
#undef B6
};
```

- Або так:

```
BitsSetTable256[0] = 0;
for (int i = 0; i < 256; i++)
{
  BitsSetTable256[i] = (i & 1) + BitsSetTable256[i / 2];
}
```

- Використовуємо (увага – вважаючи int 32-бітним):

```
unsigned int v; // Рахуємо, скільки тут одиничних біт
unsigned int cnt; // Результат

cnt = BitsSetTable256[v & 0xff] +
      BitsSetTable256[(v >> 8) & 0xff] +
      BitsSetTable256[(v >> 16) & 0xff] +
      BitsSetTable256[v >> 24];

unsigned char * p = (unsigned char *) &v;
cnt = BitsSetTable256[p[0]] +
      BitsSetTable256[p[1]] +
      BitsSetTable256[p[2]] +
      BitsSetTable256[p[3]];
```

- За більш вишуканими методами – див. оригінальне посилання, `http://graphics.stanford.edu/~seander/bithacks.html#CountBitsSet64` і далі.
- Код типу наведеного вище варто загорнути в шаблон C++, який автоматично враховуватиме ширину та працюватиме для більших за int типів.

- Варіант C++ коду, що видається оптимальним з-поміж випробуваних.

```
class bit_counter{
        static constexpr std::array<unsigned char, 256> BitsSetTable25
#   define B2(n) n,     n+1,     n+1,     n+2
#   define B4(n) B2(n), B2(n+1), B2(n+1), B2(n+2)
#   define B6(n) B4(n), B4(n+1), B4(n+1), B4(n+2)
            B6(0), B6(1), B6(1), B6(2)
#undef  .....   // Для економії місця, не наводжу
    };
public:
    template< typename T, int idx = 0,
        typename = std::enable_if_t<std::is_integral<T>::value> >
    static constexpr int count2(T arg){
        static_assert(CHAR_BIT == 8,
                      "Tell us about system you are using!");
        constexpr int bytes = sizeof(T);
        if constexpr ( idx == bytes )
            return 0;
        else
            return count2<T, idx+1>(arg)
                + BitsSetTable256[ (arg >> idx*8) & 0xff ];
    }
```

- Інші варіанти реалізації на C++, разом із аналізом, див. `Projects/cpp1_app_bit_ops/bit_counters.hpp`.

Обчислення парності

- Підрахунок в стилі Кернігана:

```
unsigned int v;          // Вхідні дані
bool parity = false;

while (v){
    parity = !parity;
    v = v & (v - 1);
}
```

Обчислення парності

- З використанням хитрої магії, для одного байта, але з множенням і остачею:

```
bool parity =
  (((b * 0x0101010101010101ULL) & 0x8040201008040201ULL)
  % 0x1FF) & 1;
```

- Для 32 бітного числа, тільки множення:

```
v ^= v >> 1;
v ^= v >> 2;
v = (v & 0x11111111U) * 0x11111111U;
bool parity = (v >> 28) & 1;
```

- Для 64 бітного числа, тільки множення:

```
v ^= v >> 1;
v ^= v >> 2;
v = (v & 0x1111111111111111UL) * 0x1111111111111111UL;
bool parity = (v >> 60) & 1;
```

- З використанням таблиці:

```
static const bool ParityTable256[256] = {
#   define P2(n) n, n^1, n^1, n
#   define P4(n) P2(n), P2(n^1), P2(n^1), P2(n)
#   define P6(n) P4(n), P4(n^1), P4(n^1), P4(n)
    P6(0), P6(1), P6(1), P6(0)
};

unsigned char b;  // Для байта -- все просто.
bool parity = ParityTable256[b];

unsigned int v;    // Для бітного 32- числа трохи важче.
v ^= v >> 16;
v ^= v >> 8;
bool parity = ParityTable256[v & 0xff];

unsigned char * p = (unsigned char *) &v; // Або так
parity = ParityTable256[p[0] ^ p[1] ^ p[2] ^ p[3]];
```

- Детальніше див. `http://graphics.stanford.edu/~seander/bithacks.html#ParityNaive`.

Зміна порядку біт на зворотній

- Прямолінійно:

```
unsigned int v;
unsigned int res = v;
int s = sizeof(v) * CHAR_BIT - 1;

for (v >>= 1; v; v >>= 1){
  r <<= 1;
  r |= v & 1;
  s--;
}
r <<= s; // досуваємо"", коли старші біти v нульові
```

- Із використанням таблиці – на наступному слайді, подробиці та інші методи: `http://graphics.stanford.edu/~seander/bithacks.html#BitReverseObvious`.

За допомогою таблиці

```
static const unsigned char BitReverseTable256[256] = {
#   define R2(n)     n,     n + 2*64,     n + 1*64,     n + 3*64
#   define R4(n) R2(n), R2(n + 2*16), R2(n + 1*16), R2(n + 3*16)
#   define R6(n) R4(n), R4(n + 2*4 ), R4(n + 1*4 ), R4(n + 3*4 )
    R6(0), R6(2), R6(1), R6(3)
};
unsigned int v; // обертатимемо по 8 біт за раз
unsigned int res;

res = (BitReverseTable256[ v         & 0xff] << 24) |
      (BitReverseTable256[(v >> 8)  & 0xff] << 16) |
      (BitReverseTable256[(v >> 16) & 0xff] << 8)  |
      (BitReverseTable256[(v >> 24) & 0xff]);

// Або так
unsigned char * p = (unsigned char *) &v;
unsigned char * q = (unsigned char *) &res;
q[3] = BitReverseTable256[p[0]];
q[2] = BitReverseTable256[p[1]];
q[1] = BitReverseTable256[p[2]];
q[0] = BitReverseTable256[p[3]];
```

Остача від ділення на "1 << s"

- Іноді корисно відкинути всі біти, старші за даний. Це можна представити як остачу від ділення на число із одним встановленим бітом.
- Проста реалізація:

```
unsigned int val = 0b1101;
unsigned int bit_number = 3;
unsigned int den = 1U << bit_number ; // Ділимо на 1, 2, 4 і тд..
unsigned int res = val & (den - 1);
```

- Тут також є спосіб, як без оператора ділення поділити на `(1 << n) - 1` – число із всіма бітами, із номером, меншим за певний, рівними 1: `http://graphics.stanford.edu/~seander/bithacks.html#ModulusDivisionEasy`

Заокруглення до наступної степені двійки

- **unsigned int** v;

```
v--;
v |= v >> 1;
v |= v >> 2;
v |= v >> 4;
v |= v >> 8;
v |= v >> 16;
v++;
```

- Якщо не робити ++ – отримаємо всі біти, із номерами, меншими або рівними старшому біту, встановлені в одиницю. Це іноді називають bit folding.
- Враховуючи, що результат такого перетворення – це `1U << (log2(v - 1) + 1)`, його можна використати для знаходження цілої частини логарифму при основі два – достатньо знайти номер встановленого біта. Або навпаки – знаючи логарифм значення, можна знайти номер старшого біта.

Розрахунок цілої частини $log_2(x)$

- Власне, ціла частина[6] такого логарифму – номер найстаршого біта.
- Найпростіший спосіб – рахуємо:

```
unsigned int val = 0b111;
unsigned int res = 0; // log2(val)

while (val >>= 1){
    ++res;
}
```

- Одне із можливих використань – знати, скільки символів треба для текстового представлення даного числа.

[6] Тобто, із заокругленням вниз.

Табличний підхід:

```
#define LT(n) n, n, n, n, n, n, n, n, n, n, n, n, n, n, n, n
static const char LogTable256[256] = {
    -1, 0, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3,
    LT(4), LT(5), LT(5), LT(6), LT(6), LT(6), LT(6),
    LT(7), LT(7), LT(7), LT(7), LT(7), LT(7), LT(7), LT(7)
};
#undef LT

unsigned val;
unsigned res;       // log_2(val)
unsigned int temp1, temp2;

if ( (temp2 = val >> 16) ){
  res = (temp1 = temp2 >> 8) ?
            24 + LogTable256[temp1] : 16 + LogTable256[temp2];
}
else {
  res = (temp1 = val >> 8) ? 8 + LogTable256[temp1] :
            LogTable256[val];
}
```

- Безпосередня генерація таблиці (але навіть сучасні компілятори не завжди справляються із інлайненням та обчисленням під час компіляції такого коду)

```
LogTable256[0] = LogTable256[1] = 0;
for (int i = 2; i < 256; i++)
{
  LogTable256[i] = 1 + LogTable256[i / 2];
}
LogTable256[0] = -1; // log_2(0) == -1
```

- Якщо аргумент, var, є степенем двійки, 2^N, можна також так:

```
unsigned int val;
constexpr unsigned int magic[] = {0xAAAAAAAA, 0xCCCCCCCC, 0xF0F0
                                  0xFF00FF00, 0xFFFF0000};
unsigned int res = (val & magic[0]) != 0;
for (int i = 4; i > 0; i--)
{
  res |= ((val & magic[i]) != 0) << i;
}
```

- Ще кілька цікавих прикладів обчислення цілої частини $log_2(val)$ є за посиланням, звідки взято опис більшості наведених вище трюків: `http://graphics.stanford.edu/~seander/bithacks.html#IntegerLog`.
- Ремарка: за тим посиланням часто пропонують розгортати цикли (loop unrolling) вручну. Для сучасних компіляторів це не обов'язково, а іноді – погана ідея, вони змушені згортати таке ручне розгортання назад, щоб потім згорнути по своєму. Також, автор іноді пише register, що вже дуже давно позбавлено сенсу.

Розрахунок цілої частини $log_{10}(x)$

- Аналогічно до логарифму пр основі 2, логарифм при основі 10 дозволяє знайти кількість десяткових цифр числа[7].
- Найпростіший спосіб:

```
unsigned int val = 123; // Не повинно бути нулем
assert(val != 0);
int res;

res = (val >= 1'000'000'000) ? 9 : (val >= 100'000'000) ?
  8 : (val >= 10'000'000) ? 7 : (val >= 1'000'000) ?
  6 : (val >= 100'000) ? 5 : (val >= 10'000) ?
  4 : (val >= 1000) ? 3 : (val >= 100) ? 2 : (val >= 10) ? 1 : 0;
```

- Добре працює, якщо числа рівномірно розподілені в 32-бітному інтервалі – абсолютна більшість буде опрацьована першою ж умовою.

[7] Не забудьте додати 1 – заокруглює вниз!

- Враховуючи, що $log_{10}(val) = \frac{log_2(val)}{log_2(10)}$, а $\frac{1}{log_2(10)} \approx \frac{1233}{4096} = \frac{1233}{2^{12}}$, можна зробити так:

```
unsigned int val = 1234;
int res;
int temp;

constexpr unsigned int const PowersOf10[] =
                    {1, 10, 100, 1'000, 10'000, 100'000,
            1'000'000, 10'000'000, 100'000'000, 1'000'000'000};

temp = (IntegerLogBase2(val) + 1) * 1233 >> 12;
res = temp - (val < PowersOf10[temp]);
```

- IntegerLogBase2() – функція, реалізована, як описано вище.
- +1 потрібен, так як заокруглює вниз.
- Так як множення на $\frac{1233}{4096}$, все ж, наближене, temp може помилятися на одиницю, мусимо підправляти[8].

[8] v < PowersOf10[t] – булева величина, яка при арифметичних операціях дасть 0 або 1

Неперервні блоки нулів праворуч

- Знаходження найстаршого біту – підрахунок кількості нульових біт ліворуч від нього.
- Іноді цікава протилежна задача – знаходження наймолодшого не нульового біта, підрахунок нулів праворуч.
- Простий спосіб – робимо всі нульові біти одиничними, одиничні нульовими, і рахуємо:

```
unsigned int val = 0b101000;
int res;
if (val){
        // Нулі праворуч – в 1, решта –– зануляємо
    val = (val ^ (val - 1)) >> 1;
    for (res = 0; val; res++){
        val >>= 1;
    }
} else{
        res = CHAR_BIT * sizeof(val);
}
```

- Ще один спосіб:

```
unsigned int val = 0b101000;
unsigned int res = 32;

val &= -static_cast<signed int>(val);
if (val) res--;
if (val & 0x0000FFFF) res -= 16;
if (val & 0x00FF00FF) res -= 8;
if (val & 0x0F0F0F0F) res -= 4;
if (val & 0x33333333) res -= 2;
if (val & 0x55555555) res -= 1;
```

- Більше прикладів – див. `http://graphics.stanford.edu/~seander/bithacks.html#ZerosOnRightBinSearch`.

Чергування біт із різних джерел

- Іноді потрібно зберегти два числа поруч. Наприклад, координати x та y, може бути потрібно запхати в один unsigned int. Один із варіантів – поділити число на половину, кожну частину вважаючи меншим цілим числом і добувати звідти частини (див. вище способи) для подальшої роботи.
- Альтернативний підхід – чергувати біти, один з x, один із y. Іноді це може бути зручно – не виникає ситуація, коли координати y завжди "більші" за координати x.
- Отримане таким чином числа можна порівнювати між собою – близькі точки, зазвичай, матимуть і близькі координати.
- Простий спосіб:

```
unsigned short x = 0b00100100;
unsigned short y = 0b10101001;
unsigned int  res = 0;

for (int i = 0; i < sizeof(x) * CHAR_BIT; i++)
{
    res |= (x & 1U << i) << i | (y & 1U << i) << (i + 1);
}
```

- Хитріші підходи – див. `http://graphics.stanford.edu/~seander/bithacks.html#InterleaveTableLookup`.

Перевірка, чи є нульові байти у слові

- Довідатися, чи є в змінній нульові байти важливо для оптимізації роботи із C-стрічками – щоб не ітерувати побайтово.
- Кілька підходів:

```
unsigned int val = 0xAB01CDFA;
bool hasZeroByte, hasNoZeroByte;

// Тривіальний підхід. Хоча, компілятор має шанс векторизувати ...
unsigned char * p = (unsigned char *) &val;
hasNoZeroByte = *p && *(p + 1) && *(p + 2) && *(p + 3);

hasZeroByte = ~((((val & 0x7F7F7F7F) + 0x7F7F7F7F) | val) |
                                                0x7F7F7F7F);

hasNoZeroByte = ((val & 0xFF) && (val & 0xFF00) &&
                 (val & 0xFF0000) && (val & 0xFF000000));

hasZeroByte = ((val) - 0x01010101UL) & ~(val) & 0x80808080UL;
```

- Подробиці та інші варіанти байтового аналізу (рівні якомусь ненульовому значенню, менші чи більші за певне значення, знаходяться в заданому інтервалі), див. http:

Посилання

- `https://fgiesen.wordpress.com/2024/10/23/zero-or-sign-extend/` – "Zero or sign extend".
- "A leap year check in three instructions" – мила оптимізація та цікавий підхід до процесу, див. також z3.

- Сучасні процесори, зазвичай, мають широкий набір інструкцій для бітових маніпуляцій.

x86

- Вже оригінальний 8086 мав велику кількість побітових операцій. Наприклад, команди AND, OR, XOR, хоч і називаються "logical AND"і т.д., виконують побітові операції, бітових зсувів взагалі неміряно: SAL, SAR, SHL, SHR, і т.д. і т.п..
- Новіші – 286, 386, і т.д., додали й більш спеціалізовані. До прикладу: BSF – "Bit Scan Forward", пошук наймолодшого одиничного біта; LZCNT – "Count the Number of Leading Zero Bits", те ж, з протилежного кінця; POPCNT – "Return the Count of Number of Bits Set to 1" і багато-багато інших.
- SIMD розширення додали аналогічні команди для відповідних широких регістрів[9].

[9] Хоча, із нюансами: https://mischasan.wordpress.com/2011/04/04/what-is-sse-good-for-2-bit-vector-operations/.



ARM



AVR



Інтринсики

- Використання асемблеру (як окремими модулями, так і за допомогою асемблерних вставок) часто незручне.
- Тому компілятори підтримують велику кількість так званих інтринсик, (intrinsic), вбудованих функцій, які, фактично, не є функціями – вони вказують компілятору вставити певну машинну команду, або ще якусь конструкцію, що не має прямого відображення в C/C++.
- Звичайно, інтринсики не портабельні між компіляторами. Однак, вони підвищують портабельність між host-системами.

- Як приклад, GCC має інтринсику `__builtin_popcount(unsigned int)` – підрахунок кількості одиничних бітів у слові. Якщо процесор підтримує відповідну команду, POPCNT, або інші ефективні способи їх порахувати – він використає їх. Якщо ні – використає чисто програмну реалізацію. Детальніше див. `https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html`.
- MSVC аналогічну вбудовану функцію називає `__popcnt(unsigned int)`, див. `https://docs.microsoft.com/ru-ru/cpp/intrinsics/popcnt16-popcnt-popcnt64?view=vs-2019`.
- З іншого боку, скажімо, інтринсики SSE задаються Інтел, тому менш більш сумісні між gcc, MSVC, icc, clang.